

The background is a light blue gradient with several realistic water droplets of various sizes scattered across it. The droplets have highlights and shadows, giving them a 3D appearance. They are concentrated more in the top-left and bottom-right corners.

DATABASE FUNDAMENTALS – CLOUD SQL

GOOGLE CLOUD PLATFORM

Databases Primer

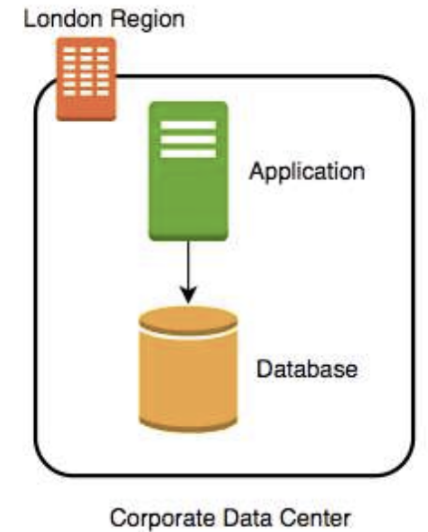
- Databases provide **organized** and **persistent** storage for your data
- To **choose between different database types**, we would need to understand:
 - Availability
 - Durability
 - RTO
 - RPO
 - Consistency
 - Transactions etc
- Let's get started on a **simple journey** to understand these



Database

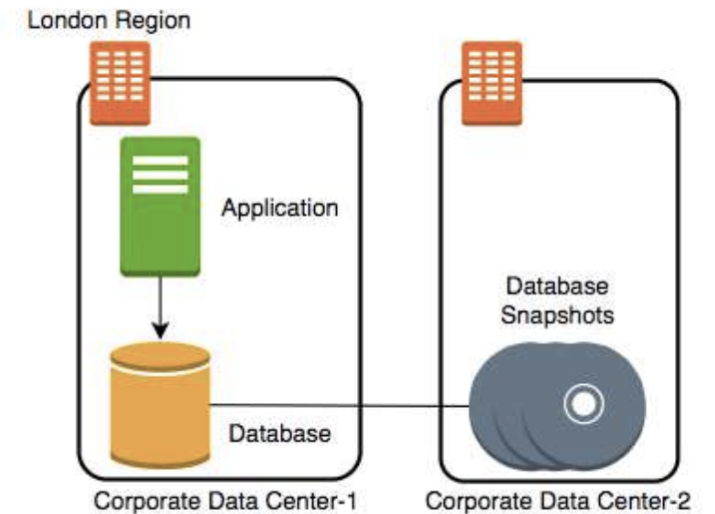
Database - Getting Started

- Imagine a database deployed in a data center in **London**
- Let's consider some challenges:
 - **Challenge 1:** Your database will go down if the data center crashes or the server storage fails
 - **Challenge 2:** You will lose data if the database crashes



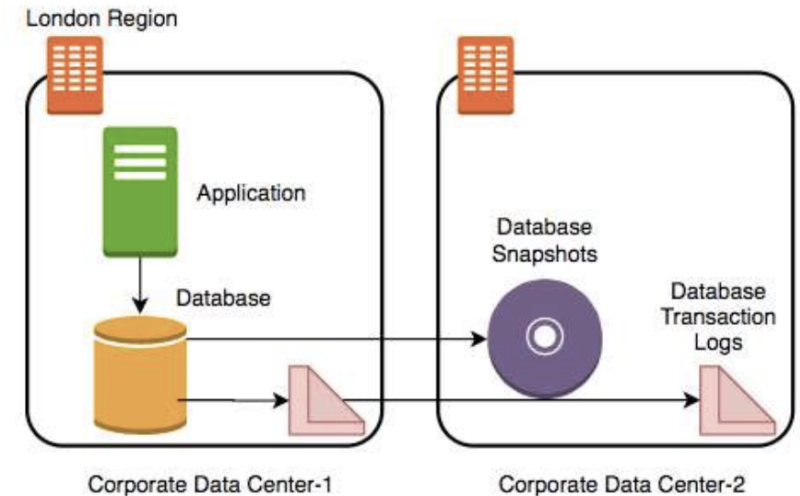
Database - Snapshots

- Let's automate taking copy of the database (**take a snapshot**) every hour to another data center in London
- Let's consider some challenges:
 - **Challenge 1:** Your database will go down if the data center crashes
 - **Challenge 2 (PARTIALLY SOLVED):** You will lose data if the database crashes
 - You can setup database from latest snapshot. But depending on when failure occurs you can lose up to an hour of data
 - **Challenge 3(NEW):** Database will be slow when you take snapshots



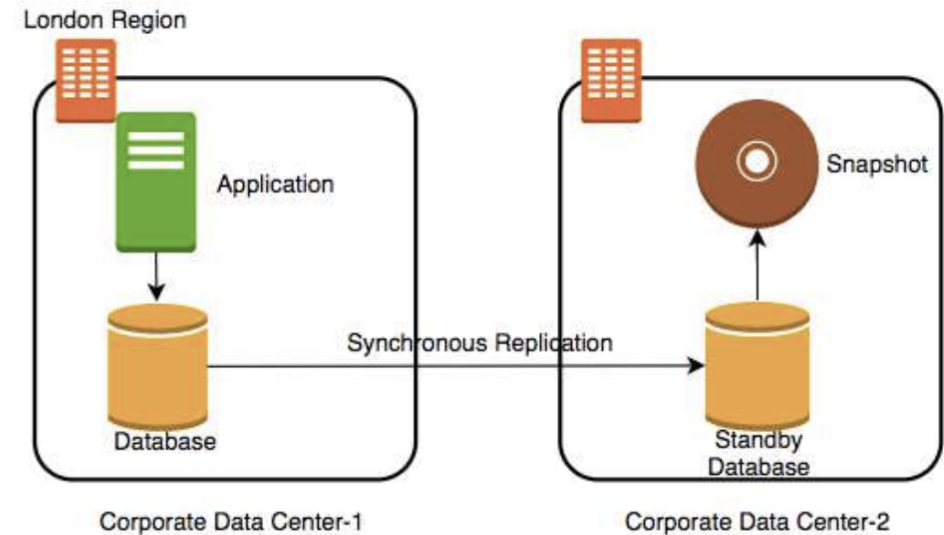
Database - Transaction Logs

- Let's add **transaction logs** to database and create a **process to copy it over** to the second data center
- Let's consider some challenges:
 - **Challenge 1:** Your database will go down if the data center crashes
 - **Challenge 2 (SOLVED):** You will lose data if the database crashes
 - You can setup database from latest snapshot and apply transaction logs
 - **Challenge 3:** Database will be slow when you take snapshots



Database - Add a Standby

- Let's add a **standby database** in the second data center with replication
- Let's consider some challenges:
 - **Challenge 1 (SOLVED):** Your database will go down if the data center crashes
 - You can switch to the standby database
 - **Challenge 2 (SOLVED):** You will lose data if the database crashes
 - **Challenge 3 (SOLVED):** Database will be slow when you take snapshots
 - Take snapshots from standby.
 - Applications connecting to master will get good performance always



Availability and Durability

- **Availability**

- Will I be able to access my data now and when I need it?
- Percentage of time an application provides the operations expected of it

- **Durability**

- Will my data be available after 10 or 100 or 1000 years?

- **Examples of measuring availability and durability:**

- 4 9's - 99.99
- 11 9's - 99.9999999999

- Typically, an **availability of four 9's** is considered very good
- Typically, a **durability of eleven 9's** is considered very good



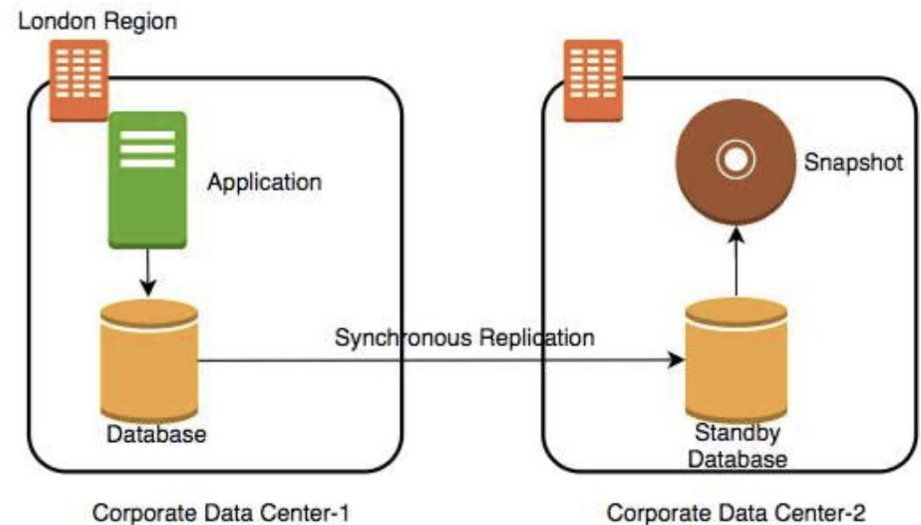
Availability

Availability	Downtime (in a month)	Comment
99.95%	22 minutes	
99.99% (4 9's)	4 and 1/2 minutes	Typically online apps aim for 99.99% (4 9's) availability
99.999% (5 9's)	26 seconds	Achieving 5 9's availability is tough



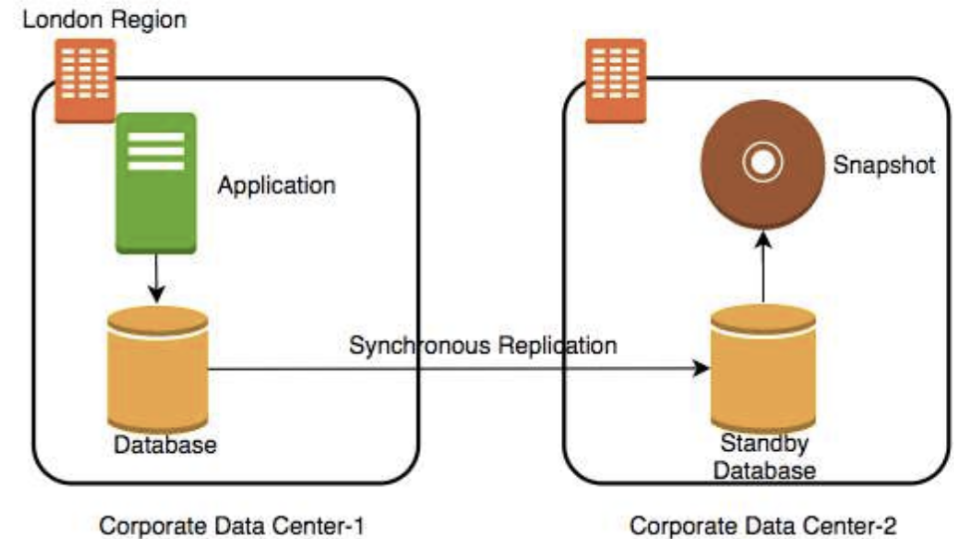
Durability

- What does a durability of 11 9's mean?
 - If you store one million files for ten million years, you would expect to lose one file
- Why should durability be high?
 - Because we hate losing data
 - Once we lose data, it is gone



Increasing Availability and Durability of Databases

- **Increasing Availability:**
 - Have multiple standbys available OR distribute the database
 - in multiple Zones
 - in multiple Regions
- **Increasing Durability:**
 - Multiple copies of data (standbys, snapshots, transaction logs and replicas)
 - in multiple Zones
 - in multiple Regions
- **Replicating data** comes with its own challenges!
 - We will talk about them a little later



Database Terminology : RTO and RPO

- Imagine a **financial transaction** being lost
- Imagine a **trade** being lost
- Imagine a **stock exchange** going down for an hour
- **Typically** businesses are fine with some downtime but they hate losing data
- Availability and Durability are technical measures
- How do we measure **how quickly we can recover from failure?**
 - **RPO (Recovery Point Objective):** Maximum acceptable period of data loss
 - **RTO (Recovery Time Objective):** Maximum acceptable downtime
- Achieving **minimum RTO and RPO is expensive**
- **Trade-off** based on the criticality of the data



Database

Question - RTO and RPO

- You are running an application in VM instance storing its data on a persistent data storage. You are taking snapshots every 48 hours. If the VM instance crashes, you can manually bring it back up in 45 minutes from the snapshot.

What is your RTO and RPO?

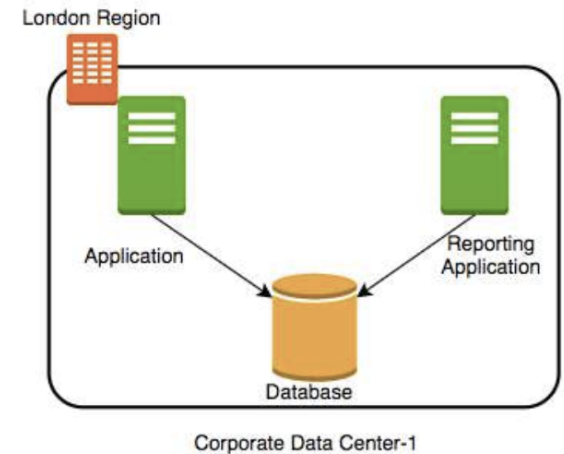
- RTO - 45 minutes
- RPO - 48 hours

Achieving RTO and RPO - Failover Examples

Scenario	Solution
Very small data loss (RPO - 1 minute) Very small downtime (RTO - 5 minutes)	Hot standby - Automatically synchronize data Have a standby ready to pick up load Use automatic failover from master to standby
Very small data loss (RPO - 1 minute) BUT I can tolerate some downtimes (RTO - 15 minutes)	Warm standby - Automatically synchronize data Have a standby with minimum infrastructure Scale it up when a failure happens
Data is critical (RPO - 1 minute) but I can tolerate downtime of a few hours (RTO - few hours)	Create regular data snapshots and transaction logs Create database from snapshots and transactions logs when a failure happens
Data can be lost without a problem (for example: cached data)	Failover to a completely new server

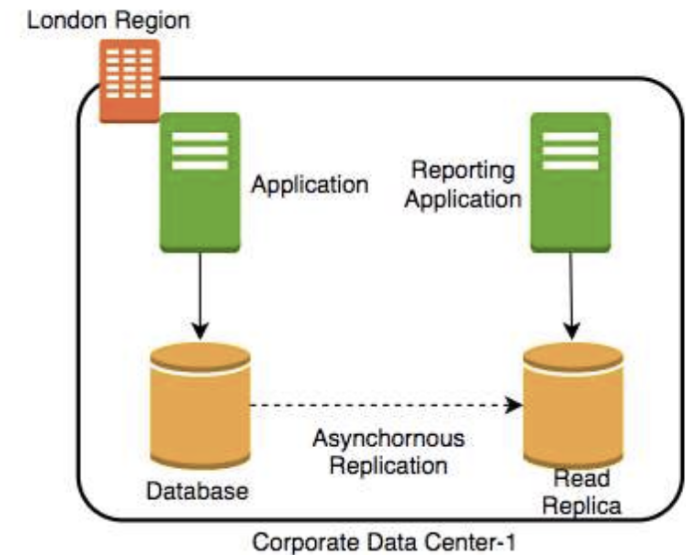
(New Scenario) Reporting and Analytics Applications

- New reporting and analytics applications are being launched using the same database
 - These applications will ONLY read data
- Within a few days you see that the database performance is impacted
- How can we fix the problem?
 - **Vertically scale the database** - increase CPU and memory
 - **Create a database cluster (Distribute the database)** - Typically database clusters are expensive to setup
 - **Create read replicas** - Run read only applications against read replicas



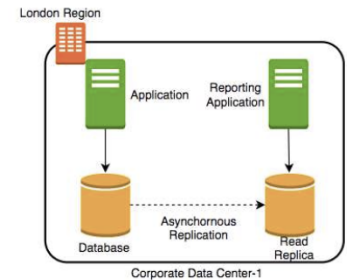
Database - Read Replicas

- Add read replica
- Connect reporting and analytics applications to **read replica**
- Reduces load on the master databases
- Upgrade read replica to master database (supported by some databases)
- Create read replicas in **multiple regions**
- **Take snapshots** from read replicas



Consistency

- How do you ensure that data in multiple database instances (standbys and replicas) is updated simultaneously?
- **Strong consistency** - Synchronous replication to all replicas
 - Will be slow if you have multiple replicas or standbys
- **Eventual consistency** - Asynchronous replication. A little lag - few seconds - before the change is available in all replicas
 - In the intermediate period, different replicas might return different values
 - Used when scalability is more important than data integrity
 - Examples : Social Media Posts - Facebook status messages, Twitter tweets, Linked in posts etc
- **Read-after-Write consistency** - Inserts are immediately available
 - However, updates would have eventual consistency



Database Categories

- There are **several categories** of databases:
 - Relational (OLTP and OLAP), Document, Key Value, Graph, In Memory among others
- **Choosing type of database** for your use case is not easy. A few factors:
 - Do you want a **fixed schema**?
 - Do you want flexibility in defining and changing your schema? (schemaless)
 - What level of **transaction properties** do you need? (atomicity and consistency)
 - What kind of **latency** do you want? (seconds, milliseconds or microseconds)
 - **How many transactions** do you expect? (hundreds or thousands or millions of transactions per second)
 - **How much data** will be stored? (MBs or GBs or TBs or PBs)
 - and a lot more...



Cloud SQL



Cloud Spanner



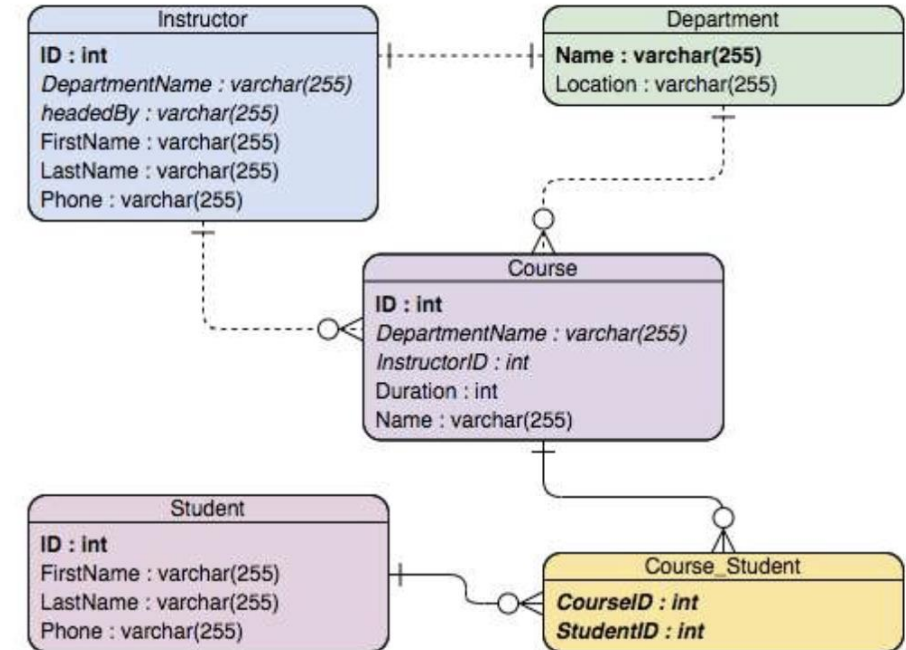
Cloud Datastore



BigQuery

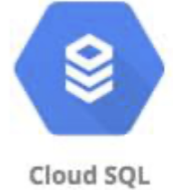
Relational Databases

- This was the **only** option until a decade back!
- Most **popular** (or unpopular) type of databases
- **Predefined** schema with tables and relationships
- **Very strong** transactional capabilities
- Used for
 - OLTP (Online Transaction Processing) use cases and
 - OLAP (Online Analytics Processing) use cases



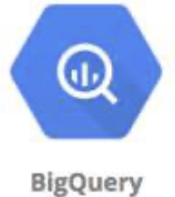
Relational Database - OLTP (Online Transaction Processing)

- Applications where large number of users make large number of small transactions
 - small data reads, updates and deletes
- **Use cases:**
 - Most traditional applications, ERP, CRM, e-commerce, banking applications
- **Popular databases:**
 - MySQL, Oracle, SQL Server etc
- **Recommended Google Managed Services:**
 - **Cloud SQL** : Supports PostgreSQL, MySQL, and SQL Server for regional relational databases (upto a few TBs)
 - **Cloud Spanner**: Unlimited scale (multiple PBs) and 99.999% availability for global applications with horizontal scaling



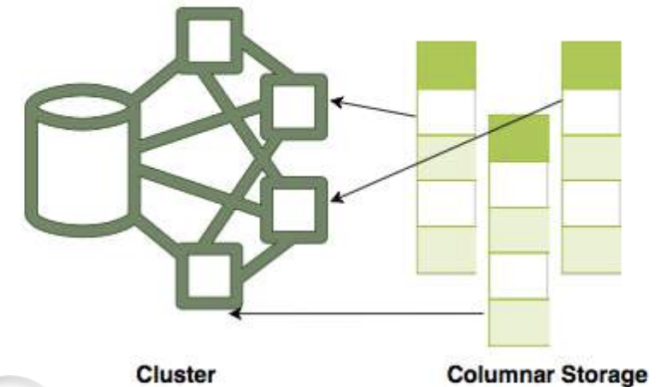
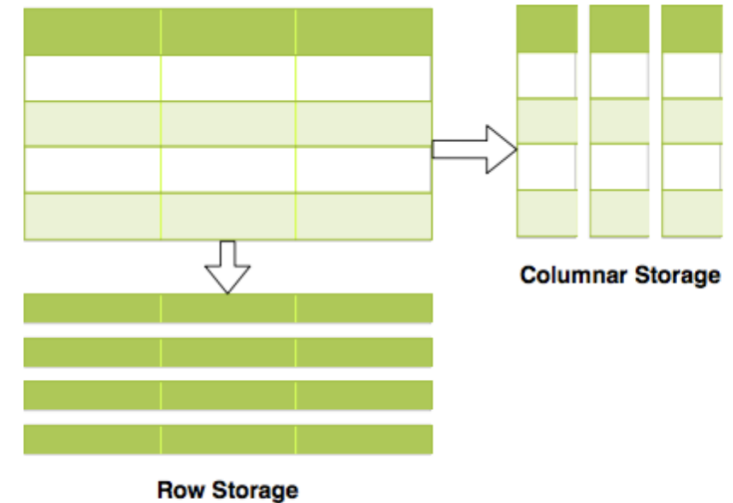
Relational Database - OLAP (Online Analytics Processing)

- Applications allowing users to **analyze petabytes of data**
 - **Examples** : Reporting applications, Data ware houses, Business intelligence applications, Analytics systems
 - **Sample application** : Decide insurance premiums analyzing data from last hundred years
 - Data is consolidated from multiple (transactional) databases
- Recommended GCP Managed Service
 - **BigQuery**: Petabyte-scale distributed data ware house

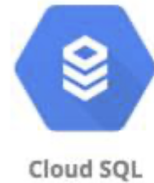


Relational Databases - OLAP vs OLTP

- OLAP and OLTP use **similar data structures**
- **BUT very different approach in how data is stored**
- **OLTP databases** use row storage
 - Each table row is stored together
 - Efficient for processing small transactions
- **OLAP databases** use columnar storage
 - Each table column is stored together
 - **High compression** - store petabytes of data efficiently
 - **Distribute data** - one table in multiple cluster nodes
 - **Execute single query across multiple nodes** - Complex queries can be executed efficiently



Cloud SQL

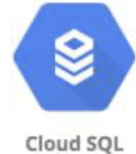


- **Fully Managed Relational Database service**
 - Configure your needs and do NOT worry about managing the database
 - Supports MySQL, PostgreSQL, and SQL Server
 - Regional Service providing High Availability (99.95%)
 - Use SSDs or HDDs (For best performance: use SSDs)
 - Upto 416 GB of RAM and 30 TB of data storage
- **Use Cloud SQL for simple relational use cases:**
 - To migrate local MySQL, PostgreSQL, and SQL Server databases
 - To reduce your maintenance cost for a simple relational database
 - (REMEMBER) Use Cloud Spanner(Expensive \$\$\$\$) instead of Cloud SQL if:
 - You have huge volumes of relational data (TBs) OR
 - You need infinite scaling for a growing application (to TBs) OR
 - You need a Global (distributed across multiple regions) Database OR
 - You need higher availability (99.999%)

Cloud SQL - Features

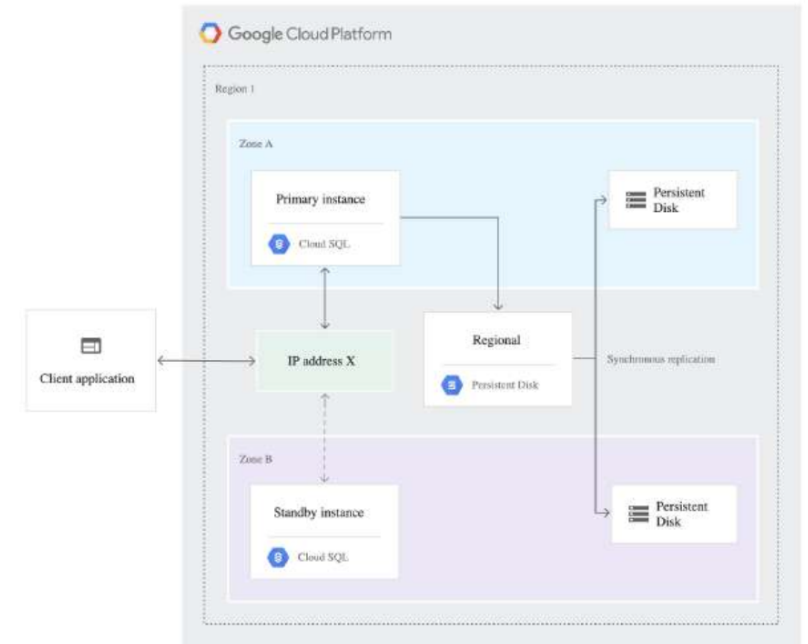
- **Important Cloud SQL Features:**

- Automatic encryption (tables/backups), maintenance and updates
- High availability and failover:
 - Create a Standby with automatic failover
 - Pre requisites: Automated backups and Binary logging
- Read replicas for read workloads:
 - Options: Cross-zone, Cross-region and External (NON Cloud SQL DB)
 - Pre requisites: Automated backups and Binary logging
- Automatic storage increase without downtime (for newer versions)
- Point-in-time recovery: Enable binary logging
- Backups (Automated and on-demand backups)
- Supports migration from other sources
 - Use Database Migration Service (DMS)
- You can export data from UI (console) or gcloud with formats:
 - SQL (Recommended if you import data into other databases) and CSV



Cloud SQL - High Availability

- Create a High Availability (HA) Configuration
 - Choose **Primary and Secondary** zones within a region
 - You will have two instances : **Primary** and **Secondary** instances
- Changes from primary are replicated **synchronously** to secondary
- In case of **Zonal** failure, automatic failover to secondary instance:
 - If **Primary zone** becomes available, failover does not revert automatically
- (Remember) **High Availability** setup CANNOT be used as a **Read Replica**



source:cloud.google.com

Cloud SQL - CommandLine

- ***gcloud sql***

- ***gcloud sql instances create/clone/delete/describe/patch***
 - *gcloud sql instances create INSTANCE*
 - *gcloud sql instances patch --backup-start-time*
- ***gcloud sql databases create/delete/describe/list/patch***
 - *gcloud sql databases create DATABASE --instance=INSTANCE*
- ***gcloud sql connect INSTANCE [--database=DATABASE --user=root]***
- ***gcloud sql backups create/describe/list***
 - *gcloud sql backups create --async --instance [INSTANCE]* (one time backup)

